Universidade da Beira Interior
Departamento de Informática

Nº 50391 - 2026 *Testing System for Java Payara APIs: An Automated Regression and CI/CD Framework for the GNSS EPOS API*

Author:

Eduardo Pires Valentim de Almeida Abrantes

Supervisor and Co-Supervisor:

Professor Paul Andrew Crocker
Vasco Senra

June 19, 2026

Acknowledgments

The completion of this project, and indeed of a large part of my academic journey, would not have been possible without the help, guidance and patience of many people. I would first like to thank my supervisor, Professor Paul Andrew Crocker, and my co-supervisor, Vasco Senra, for proposing this project and for the technical direction, the critical reviews and the trust placed in me throughout the development of the work described in this report.

A special word of gratitude goes to the staff of the SEGAL Laboratory at the Universidade da Beira Interior, Luís Carvalho and Fernando Geraldes, for the patient support with the virtual machines, the internal network and the VPN access that made the laboratory's infrastructure reachable from outside the lab and that, in practice, made the work described in this report possible.

I would also like to thank the colleagues and researchers I had the opportunity to work with at the laboratory, for the many discussions about the legacy code base, the refactoring strategy and the painful reality of regression testing when no two responses are quite alike. Those conversations shaped many of the choices presented in the following chapters.

Finally, to my family, for the constant and unconditional support that allowed me to reach this point, and to the friends and colleagues who were patient enough to listen to long monologues about differential testing, JAX-RS endpoints and YAML pipelines: thank you.

Contents

List of Figures

List of Tables

List of Code Listings

Acronyms

API	Application Programming Interface
BOM	Bill Of Materials
CDI	*Contexts and Dependency Injection*
CI	Continuous Integration
CD	Continuous Delivery
CSV	Comma-Separated Values
DSL	Domain Specific Language
EPOS	European Plate Observing System
GNSS	Global Navigation Satellite System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
ID	Identifier
JAR	Java Archive
JAX-RS	Jakarta RESTful Web Services
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
JWT	JSON Web Token
OpenAPI	OpenAPI Specification
POJO	Plain Old Java Object
REST	Representational State Transfer
RINEX	Receiver Independent Exchange Format
SEGAL	Space and Earth Geodetic Analysis Laboratory
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UI	User Interface

URL Uniform Resource Locator
VM Virtual Machine
VPN Virtual Private Network
WAR Web Application Archive
XML eXtensible Markup Language
YAML *YAML Ain't Markup Language*

Chapter

1

Introduction

1.1 Framing

This document describes the work developed in the context of the final-year project of the Computer Science and Engineering programme of the Universidade da Beira Interior (UBI), under the project code PC26-4 of the 2025/2026 academic year. The project was proposed by, and carried out at, the Space and Earth Geodetic Analysis Laboratory (SEGAL) laboratory of UBI, under the supervision of Professor Paul Andrew Crocker and the co-supervision of Vasco Senra, with day-to-day infrastructure support from Luís Carvalho and Fernando Geraldes (responsible for the laboratory's virtual machines, internal network and Virtual Private Network (VPN) access).

SEGAL is the research laboratory of UBI that operates and maintains a Global Navigation Satellite System (GNSS) data and metadata service contributing to the European Plate Observing System (EPOS), a pan-European research infrastructure focused on solid Earth science [1, 2]. As part of the EPOS Thematic Core Service for GNSS data and products, SEGAL runs a Jakarta EE Representational State Transfer (REST) Application Programming Interface (API) that exposes GNSS station metadata and observation files (in the Receiver Independent Exchange Format (RINEX) format [3]) to scientists, operators and downstream services across Europe.

The API in question has been in operation, in various forms, for several years. Over that period the code base accumulated significant technical debt: methods of five thousand lines, files of more than thirty thousand lines, duplicated logic copy-pasted across resources, and a long tail of patches added by successive developers and contributors to handle individual edge cases. A complete refactor of the API was therefore initiated, in which large parts of the

application were rewritten from scratch on top of Jakarta EE 11 and the Payara Platform [4, 5], while a small core of well tested logic was carried over from the previous version.

A refactor of this magnitude poses a very specific risk: any seemingly innocuous change in a query, a response field or a validation rule can produce subtle, hard to detect regressions that only manifest when a particular client, a particular station or a particular file is requested. Until this project, validation of those changes was carried out manually, a developer would deploy the new version, run a few requests by hand, and compare the resulting payloads against the previous one. This is precisely the situation against which this work is positioned.

1.2 Motivation

The motivation for this project is the gap between the pace at which the new API is being developed and the rate at which a human can reasonably validate that each new version behaves exactly like the previous one on the cases that already worked. Manual testing of a REST API that returns thousands of records, in several formats (JavaScript Object Notation (JSON), eXtensible Markup Language (XML), Comma-Separated Values (CSV)) and through dozens of query combinations, is both error prone and slow. Worse still, it does not scale to the cadence of a continuous refactor where commits land several times a day.

The need is twofold. On the one hand, the laboratory wanted an *automated* mechanism to compare every new build of the API against the previous one and flag behavioural divergences early, before they reach the production deployment. On the other hand, the laboratory also wanted to bring this mechanism inside a real Continuous Integration (CI)/Continuous Delivery (CD) pipeline running on infrastructure that the laboratory itself controls: a self-managed GitLab instance, paired with one or more GitLab Runners hosted inside the laboratory's network. The combination of the two is what turns a one-off testing script into a sustainable practice.

In addition to the immediate value for the EPOS API, the exercise of designing, prototyping and integrating such a framework was also seen as a vehicle for the student to gain hands-on experience with modern Jakarta EE features, with CI/CD pipelines, with self-hosted infrastructure, and with the practice of regression testing against a moving baseline, skills that are directly relevant beyond the scope of this particular project.

1.3 Objectives

In line with the project proposal, the work described in this report pursues the following objectives:

1. To design and implement an *automated regression testing framework* for a Jakarta EE REST API running on the Payara Platform, capable of comparing the behaviour of two builds of the application across multiple response formats (JSON, XML, CSV);

2. To integrate that framework into a self-managed GitLab CI/CD pipeline, with dedicated GitLab Runners hosted inside the laboratory's network and with appropriate access to the laboratory's databases over a VPN;

3. To validate the proposed approach on two prototype Jakarta EE applications, one focused on the CI/CD pipeline itself, the other focused on Jakarta EE 11 features, before applying it to the production API;

4. To produce a structured comparison between consecutive versions of the production API, both at the level of internal validation/geometry logic and at the level of Hypertext Transfer Protocol (HTTP) responses, with reports that are consumable both by a human developer and by the CI system;

5. To document the architecture, usage, limitations and operating procedure of the framework, in such a way that the laboratory can continue to use and evolve it after the end of this project.

1.4 Main Contributions

The main contributions of this work, expanded in detail in the subsequent chapters, can be summarised as:

- A reusable, three-stage GitLab CI/CD pipeline template for Jakarta EE applications deployed on Payara, with caching, artifact management and JUnit-compatible reporting;

- A code-level differential test harness (`DiffHarness.java`) that loads the current build and a baseline build of a Java class in two isolated class loaders and compares their outputs on a shared input grid; the harness is invoked twice by the pipeline, against two baselines (the immediate parent commit `HEAD~1` and a fixed reference commit whose

Identifier (ID) is provided through the `DIFF_FIXED_BASELINE` CI/CD variable), producing a true three-way comparison per pipeline run;

- A complementary HTTP-level differential test harness (`HttpDiff.java`) that boots one instance of Payara Micro per available Web Application Archive (WAR), the **HEAD!** (**HEAD!**) build and the two baselines, on different ports, replays a configurable suite of HTTP cases against every instance and emits one report per baseline so that per-commit regressions and accumulated drift remain distinguishable;

- Two prototype Jakarta EE 11 applications, a minimal "hello world" service used to harden the pipeline itself, and a small room-booking API used to evaluate Jakarta EE features in isolation, both reusable as didactic examples for future students of the laboratory.

1.5 Organization of the Document

In order to reflect the work carried out, this document is structured into six chapters, each serving a specific purpose:

- **Chapter 1 – Introduction.** Frames the project in its institutional and technical context, presents the motivation and objectives, lists the main contributions and outlines the organization of the document.

- **Chapter 2 – State of the Art.** Reviews the main families of test frameworks and CI/CD systems relevant to a Jakarta EE API, and surveys existing approaches to differential testing of services, in order to situate the proposed solution.

- **Chapter 3 – Technologies and Tools Used.** Describes the concrete stack of technologies adopted during the project: API styles and specification formats, the Java/Jakarta ecosystem, Structured Query Language (SQL) databases, application servers, version control, containerization and networking.

- **Chapter 4 – Methodology and Architecture.** Explains the iterative methodology, two prototype applications followed by the production application, and the overall architecture of the testing framework, with particular focus on the first prototype (`java-ee-app`) and the CI/CD patterns that were validated on it, as well as the second prototype (`syncspace-jakarta-app`) used for Jakarta EE 11 feature exploration.

- **Chapter 5 – Use Case: EPOS API.** Describes the application of the methodology to the production API (`api_v4.0`): the EPOS context, the structure of the application, the three-stage CI/CD pipeline, the two differential harnesses and the corresponding reports.
- **Chapter 6 – Conclusions and Future Work.** Summarises the results, reflects on what was learned during the project and proposes directions for future work, in particular the items left out of scope.

1.6 Repository Access

The source code of all three applications discussed in this report is hosted on GitLab. The production API (`api_v4.0`) [1] is maintained inside the `segalubi` group and access is restricted to laboratory members. The two prototype applications are published in the author's personal namespace and can be consulted publicly:

- `java-ee-app`: `https://gitlab.com/PatalJunior/java-ee-app`
- `syncspace-jakarta-app`: `https://gitlab.com/PatalJunior/syncspace-jakarta-app`

Each repository contains a `README.md` with build instructions, and a `.gitlab-ci.yml` file (where applicable) describing the pipeline discussed in Chapters 4 and 5.

[1] `https://gitlab.com/segalubi/api_v4.0`

Chapter

2

State of the Art

2.1 Introduction

This chapter reviews the body of practice and tooling relevant to the problem addressed by this project: how to automatically validate a Jakarta EE REST API, build after build, against a moving reference. It is organised around three axes. Section 2.2 surveys the main families of test frameworks used in the Java world, from low-level unit testing to in-container and HTTP-level testing. Section 2.3 discusses the CI/CD platforms that are commonly used to orchestrate those tests, with particular focus on GitLab, which is the platform used by the SEGAL laboratory and which directly constrains the design described later in the report. Section 2.2.5 reviews differential and regression testing approaches that influenced the design of the harnesses presented in Chapter 5. Section 2.4 closes the chapter.

2.2 Test Frameworks for the Java Ecosystem

The Java test landscape can be roughly partitioned into four layers: unit testing, integration testing, container testing and API/ HTTP testing. Each layer answers a different question about the system under test, and modern projects typically combine several of them.

2.2.1 Unit Testing: JUnit and TestNG

JUnit 5 [6] is the de facto standard for unit testing in Java. Its modular architecture, *Jupiter* as the programming/extension model, *Vintage* for backward compatibility with JUnit 4, and the *Platform* as the runner, allows

it to host a wide variety of test engines and to integrate with virtually every build tool and Integrated Development Environment (IDE). The `@Test`, `@ParameterizedTest` and `@Nested` annotations, together with a rich set of *lifecycle* and *extension* hooks, cover the vast majority of unit testing needs.

TestNG [7] is the historical alternative. While its adoption has declined in greenfield projects since the release of JUnit 5, it still has advantages in scenarios that require sophisticated test ordering and parallel execution out of the box, and remains common in large enterprise code bases.

For this project, JUnit 5 was chosen because of its first-class support in Maven Surefire/Failsafe, its native integration with the Arquillian Jupiter container and the simplicity of running it inside a GitLab pipeline, which natively understands the JUnit XML reporting format.

2.2.2 Integration Testing: Maven Failsafe and Testcontainers

Unit tests answer the question "does this class behave correctly in isolation". Integration tests answer the question "do these classes behave correctly when wired together with their real collaborators". In a Maven-based Jakarta EE project, integration tests are conventionally named `*IT.java` and executed by the *Failsafe* plugin during the `verify` phase, which is distinct from the `test` phase used by Surefire for unit tests.

Testcontainers [8] is the most widely adopted solution for spinning up real infrastructure, databases, message brokers, browsers, application servers, inside Docker containers purely for the duration of a test run. It is increasingly used as a replacement for in-memory fakes (such as H2 acting as PostgreSQL), because the parity between test and production environments is much higher. Testcontainers was considered for the prototypes described in Chapter 4 but ultimately not adopted, because the production database fixture is hosted inside the laboratory's network and is already reachable from the GitLab Runner via the VPN; a second, containerised copy would only complicate the picture.

2.2.3 Container Testing: Arquillian

Arquillian [9] is a container testing framework that started in the Java EE world and has since followed the rename to Jakarta EE. Its key idea is the *micro-deployment*: each test method ships its own small WAR/Java Archive (JAR), packaged in code through *ShrinkWrap*, that is deployed into a real container (managed, embedded or remote) just before the test runs and undeployed immediately after. This allows Jakarta RESTful Web Services (JAX-RS) resources, *Contexts and Dependency Injection* (CDI) beans

and Jakarta Persistence API (JPA) entities to be tested with the same lifecycle and the same configuration as in production, while avoiding the cost of maintaining a fully populated server.

Arquillian was used in the first prototype (Section 4.4) to validate the CI/CD pipeline, but was deliberately *not* adopted for the production API. Two reasons motivated this choice: first, the production application is far too large to benefit from per-test micro-deployments; and second, the laboratory's real need was not to test a single deployment in isolation, but to compare *two* deployments of slightly different code at the same time, which is more naturally expressed by spawning two Payara Micro processes in parallel.

2.2.4 API-level Testing: REST Assured, Postman/Newman

REST Assured [10] is a Java Domain Specific Language (DSL) for testing REST services, with a fluent "`given()....when()....then()`" syntax heavily inspired by behaviour-driven development. It is the most common choice for HTTP-level tests written in the same language as the system under test, and it integrates with JUnit 5 transparently. It was considered as an alternative to writing a custom HTTP harness with `java.net.http.HttpClient`, but in the end the *differential* nature of the tests, the same request, sent to two endpoints, with two responses to be diffed, mapped more naturally onto plain `HttpClient` calls.

Postman [11] is a graphical API client widely used by developers and **QA!** (**QA!**) teams to manually exercise endpoints. Its companion command-line runner **Newman** [12] can replay Postman *collections* (JSON files describing a set of requests and assertions) inside a CI pipeline. This is an attractive solution for teams in which the API clients are maintained by non-developers, but its assertion model (JavaScript-based, embedded in the collection) makes diffing two responses against each other awkward.

2.2.5 Differential Testing

A specific family of testing approaches that directly inspired the work described later in this report is *differential testing*. Rather than asserting that the system under test produces a specific expected response, differential testing relies on the existence of a "reference" implementation (or version) and asserts only that the two implementations agree on a shared set of inputs.

The canonical published example in the service world is **Diffy** [13], a proxy released by Twitter Engineering that sits in front of a candidate service and a primary service and replays each incoming request against both, reporting any divergence in the responses. Diffy and its descendants are designed for

the case in which a new version of a service must be validated by comparing its behaviour against the version it is about to replace, exactly the case faced by the SEGAL laboratory during the refactor of its API.

A second family of approaches is so-called *snapshot testing*, popularised in the JavaScript world by Jest and now common in other ecosystems. Snapshot testing serialises the output of a piece of code to a file the first time the test runs, and on every subsequent run asserts that the new output matches the stored snapshot. It scales poorly when the output of the system depends on a live database or when the legitimate response can vary slightly between runs, both of which are true for the EPOS API.

The approach taken in this project is closer to Diffy in spirit than to snapshot testing: rather than freezing a single reference, the reference is always the parent commit of the build currently under test, which keeps the comparison meaningful across long refactor branches without requiring a human to update fixtures.

2.3 Continuous Integration and Continuous Delivery

2.3.1 Foundations

The notion of CI/CD as articulated by Humble and Farley [14] is the practice of integrating code into a shared mainline frequently, typically several times a day, and of keeping that mainline in a state from which a deployable artifact can be produced at any time, through a fully automated pipeline. The pipeline is canonically organised into a small number of stages, build, test, deploy, with strict gating between them: no stage runs unless the previous one has succeeded, and a failure stops the pipeline and surfaces a clear signal to the team.

The same publication popularised the idea of the *deployment pipeline* as the single source of truth about a build's quality. In a mature setup, the artifact produced by the `build` stage is the exact same artifact that flows through the test stages and eventually to deployment, ensuring there is no drift between what was tested and what was shipped. The same principle is applied in this project: the WAR built once in the first stage of the pipeline is consumed verbatim by the testing stages later on.

2.3.2 Platforms

Three platforms dominate the modern CI/CD space and are worth comparing in the context of the laboratory's choice.

Jenkins [15] is the long-standing, plugin-based, self-hosted automation server. Its strengths are flexibility and breadth of plugin coverage; its main drawbacks are that pipelines are historically described in Groovy (with the related learning curve) and that the maintenance burden of the master/agent topology is non-trivial.

GitHub Actions [16] is a hosted CI/CD service that integrates tightly with repositories hosted on GitHub. Its pipelines are described in *YAML Ain't Markup Language* (YAML) and its ecosystem of reusable *actions* is large. It is a very strong default for open source projects, but it cannot be self-hosted as an end-to-end platform; the control plane runs on GitHub's infrastructure.

GitLab CI/CD [17] is the CI system integrated into GitLab. Pipelines are described in a single `.gitlab-ci.yml` file at the root of the repository; runners are separate processes that pick up jobs from the GitLab server, and can be *shared* (provided by gitlab.com) or *specific* [18], registered against a particular project or group. The feature that matters most for this project is that GitLab itself can be *self-managed* [19]: the entire control plane can be installed inside the laboratory's network, and combined with one or more locally-hosted runners can be used to build, test and deploy applications that depend on internal resources such as the laboratory's GNSS database, without exposing those resources to the public Internet.

That last property is the deciding factor. The SEGAL laboratory operates a self-managed GitLab instance and a GitLab Runner host provisioned specifically for the EPOS API; that runner is inside the laboratory's network and can reach the production database directly. The choice of GitLab CI/CD as the orchestration platform for this project is therefore not an abstract preference but a consequence of the operating environment.

2.4 Conclusion

The state of the practice in Java API testing offers a rich catalogue of tools at each layer of the stack, and a small number of mature CI/CD platforms in which to orchestrate them. From this catalogue, the project adopts JUnit 5 as the unit/integration test framework, plain `java.net.http.HttpClient` as the HTTP client for the differential harness, and GitLab CI as the orchestration platform, the last choice being effectively mandated by the laboratory's infrastructure. Differential testing, as embodied by tools like Diffy, provides

the conceptual model on top of which the bespoke harnesses described in Chapter 5 are built. The next chapter details the concrete stack of technologies on which the implementation rests.

Chapter

3

Technologies and Tools Used

3.1 Introduction

This chapter describes the concrete set of technologies and tools on which the work reported in this document is built. The set was not chosen in a single decision, but converged over the course of the project through a mixture of constraints (what the production API already uses, what the laboratory's network allows, what the proposal explicitly mandates), comparisons (between equivalent alternatives) and pragmatic preference. The chapter is structured by layer of the stack: API contracts and serialization formats in Section 3.2, the Java/Jakarta ecosystem in Section 3.3, persistence and SQL databases in Section 3.4, application and CI servers in Section 3.5, version control in Section 3.6, containerization in Section 3.7 and network access in Section 3.8.

3.2 APIs, Serialization Formats and Specification

3.2.1 REST and the Multi-format Response

All endpoints exposed by the applications discussed in this report follow the REST architectural style as described by Fielding [20]: resources are identified by stable Uniform Resource Locators (URLs), manipulated through a small set of HTTP verbs (with `GET` dominating for read-only GNSS metadata), and the API is stateless on the server side. There is no session affinity, which is a property that the differential test harness described in Chapter 5 relies on heavily: the same request can be replayed against two independent server processes and the responses can be compared directly.

The production API (`api_v4.0`) is unusual in that it serves the *same* resource in several serialization formats, selected by a path parameter at the end of the URL. The supported formats are JSON (default), XML (produced through Jackson's XML data format module), CSV (produced through a custom writer) and, for the file metadata endpoints, an executable shell *script* that downloads the corresponding files. This multi-format output was inherited from the legacy API, many of its clients are scientific scripts that consume CSV or JSON directly, and was preserved in the refactor; it is also the reason the HTTP-level differential harness has to compare not only headers and status codes but also the body, format by format. Figure 3.1 sketches the content negotiation pattern that turns a single resource method into four wire formats.

3.2.2 JSON and XML

JSON is the default serialization and the format on which most of the test fixtures of this project rely. The Jakarta REST implementation provided by Payara takes care of object serialization through Jackson, with negligible per-request overhead. XML is provided by the `jackson-dataformat-xml` module, configured to preserve the same Java models that produce JSON. This dual configuration is valuable because it removes a whole class of *schema drift* bugs: as long as the Java model is correct, both JSON and XML outputs are consistent with each other.

3.2.3 CSV

CSV is the format favoured by historical clients of the API and is produced through a small set of writer classes that flatten the entity tree into rows. The differential harness handles CSV exactly the same way it handles JSON and XML: the textual body of the response is treated as an opaque blob that must match, byte for byte, against the baseline.

3.2.4 OpenAPI Specification (OpenAPI) and Swagger User Interface (UI)

The contract of the production API is described in an OpenAPI 3.1.0 [21] document (`openapi.json`) that ships inside the WAR. The same document is served to a Swagger UI [22] instance hosted at `/swagger.html`, which provides an interactive, in-browser playground for the endpoints. Two practical benefits arise from this setup. First, the document is the single source of truth

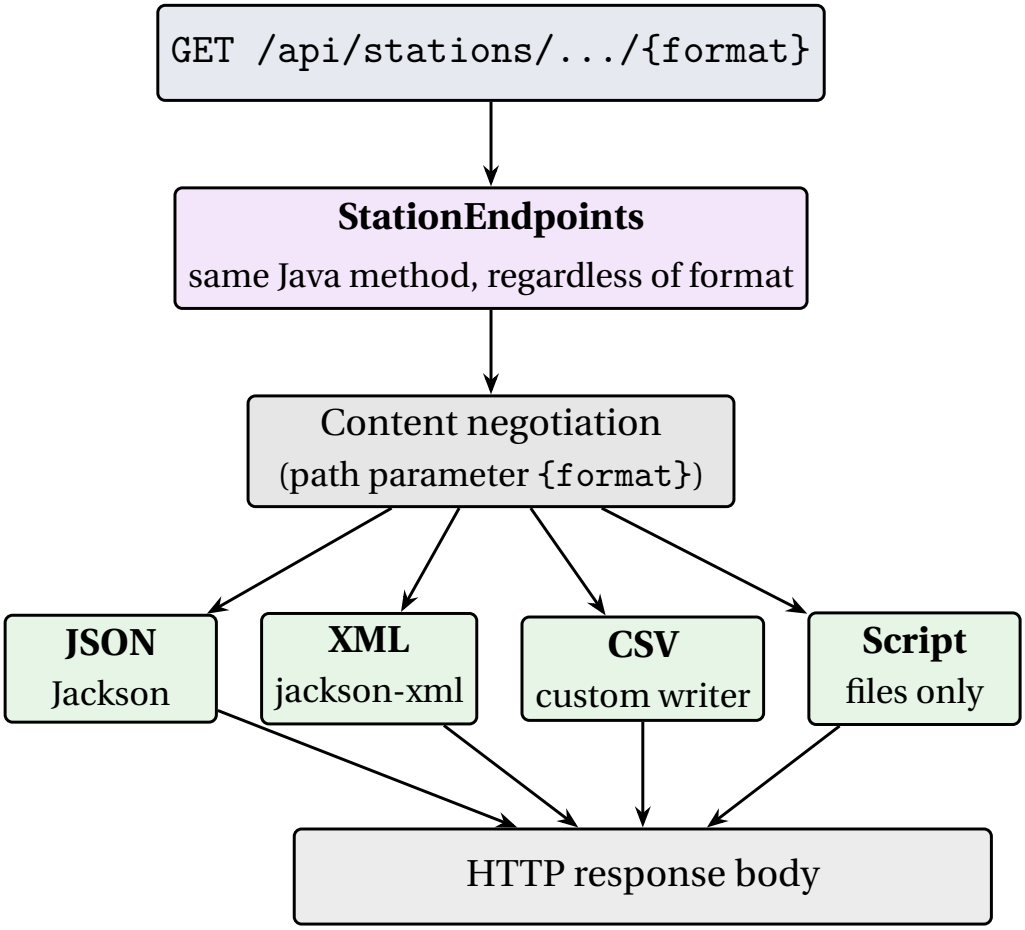

Figure 3.1: Multi-format content negotiation in `api_v4.0`. The same Java resource method is reused across four serializations, selected by a path parameter, and the differential harness must compare the body of each format separately.

shared with downstream consumers of the API, in particular the EPOS portal. Second, the same document can be used to drive contract tests: a future evolution of the test framework, briefly discussed in Chapter 6, is to derive the HTTP differential test cases automatically from the OpenAPI specification instead of maintaining them in a separate JSON file.

3.3 Java and Jakarta EE

3.3.1 The Java Platform

All applications discussed in this report run on the Java Platform. The production API and the second prototype target Java SE 21 [23], the long-term support release at the time of the project, while the first prototype is built against Java SE 17, the version against which its base CI image (`maven:3.9.6-eclipse-temurin-21`) remains backwards-compatible. The choice of Java was effectively imposed by the existing code base, but is independently justified by the maturity of the platform, the strength of its enterprise ecosystem and the very wide set of tools (IDEs, build systems, test frameworks) that target it.

The Java records introduced in recent versions of the language were used liberally in the prototypes for immutable request and response Plain Old Java Objects (POJOs), and the new `switch` expression syntax is used in several places where the refactored validation logic replaced long `if/else` chains from the legacy API.

3.3.2 Jakarta EE and the Move from Java EE

Java EE was renamed to Jakarta EE in 2018, when its stewardship moved from Oracle to the Eclipse Foundation [4]. The most visible technical consequence of the move was the change of the root package from `javax.*` to `jakarta.*`, starting with Jakarta EE 9. For an application of the size of the production API, this was not a trivial change, but it had already been absorbed by the time this project started; the current code base targets Jakarta EE 11.

The specific Jakarta specifications used in this work are summarised in Table 3.1.

Specification	Purpose	Version
Jakarta REST (JAX-RS)	HTTP endpoints, content negotiation	4.0
Jakarta CDI	Dependency injection, bean lifecycle	4.1
Jakarta Persistence (JPA)	Object-relational mapping	3.2
Jakarta Bean Validation	Declarative constraint checking	3.1
Jakarta JSON Binding (JSON-B)	POJO ↔ JSON	3.0
Eclipse MicroProfile JWT	JSON Web Token authentication	2.1
Eclipse MicroProfile Health	Liveness/readiness probes	4.0

Table 3.1: Jakarta EE and MicroProfile specifications used across the three applications.

3.3.3 Why Jakarta EE rather than Spring Boot

A common alternative to Jakarta EE for a green-field Java API is Spring Boot [24]. The two are largely equivalent at the level of features, but they differ in philosophy: Spring Boot bundles an embedded servlet container and a curated, opinionated Bill Of Materials (BOM); Jakarta EE relies on an external application server, in this case Payara, which provides the runtime contracts that the application targets.

For the prototypes, Spring Boot would have been a perfectly viable choice. For the production API, however, the existing code base is already on Jakarta EE and is deployed onto an existing Payara server inside the laboratory's network. Migrating to Spring Boot would have meant a second, parallel refactor on top of the one already in progress, with no proportional benefit. Sticking with Jakarta EE was therefore the lower-risk option.

3.4 SQL and the Database Layer

3.4.1 SQL as a Lingua Franca

The persistence model of the production API is heavily relational and the queries themselves are non-trivial: GNSS stations, networks, agencies, countries, file types, observation summaries and embargo windows are all linked through foreign keys, and most endpoints execute several joins per request. SQL [25] is therefore the natural query language for this domain; even where the application code uses JPA's object-oriented abstractions (Section 3.4.3), what reaches the database is plain SQL.

3.4.2 PostgreSQL and MySQL

The laboratory standardised on **PostgreSQL** [25] for the EPOS production database. PostgreSQL was preferred over **MySQL** [26] for two main reasons. First, PostgreSQL's adherence to the SQL standard, in particular for advanced `JOIN` types, common table expressions, the `numeric` type used for the high-precision station coordinates, and the rich set of available index types (B-tree, BRIN, GiST and others), made the rewrite of the legacy queries simpler. Second, PostgreSQL leaves the door open to the PostGIS extension for the bounding-box queries that the GNSS domain naturally invites; that extension is not in use today (see Section 3.4.4) but is a realistic future addition. MySQL was nevertheless used in the docker-compose configuration of the second prototype (Section 4.5) to validate that the application-level code is not tied to a specific dialect.

3.4.3 JPA, Hibernate and the Persistence Unit

JPA [27] is the Jakarta specification for object-relational mapping in Java. The production API uses Hibernate [28] as the JPA provider, configured through a `persistence.xml` file. The persistence unit is named `MeuPU` and is declared as `RESOURCE_LOCAL`, which means that transactions are managed explicitly by application code rather than by the container, a deliberate choice that keeps the differential test harness in full control of when transactions begin and end.

Database connections are pooled through **HikariCP** [29] with a small, conservative pool (2–15 connections, 20-second connect timeout, 30-second idle, 5-second validation), tuned for the read-mostly workload of the API and for the relatively low concurrency observed in production.

3.4.4 The EPOS Database in Practice

A closer look at the production database is useful at this point, because most of the discussion in the following chapters takes its shape and its peculiarities for granted. The schema lives in the `public` schema of a PostgreSQL 16 instance and is shared by two database fixtures, both reachable from the laboratory's internal network on port 5432 of `192.168.168.30`: `DB_3.0_J` hosts a small but representative production subset; `DB_3.0_F` hosts the full dataset and is reserved for volume-oriented and performance experiments.

Schema shape. The current schema contains forty-two tables and roughly fifty foreign-key constraints; the model is highly normalised, with no table currently exceeding thirty columns. The tables fall into four broad families:

- *Core domain*, `station`, `network`, `agency`, `country`, `rinex_file` and `highrate_rinex_file`. These hold the entities that appear in the public API payloads.
- *Junction tables*, `agency_network`, `station_network`, `station_agency_role`, `node_station` and `node_data_center`. These materialise the many-to-many relationships that the domain calls for (a station belongs to one or more networks, an agency operates one or more networks, and so on).
- *Quality-control summaries*, a family of seven tables (`qc_report_summary`, `qc_constellation_summary`, `qc_header_observations`, `qc_navigation_msg` and four typed `qc_observation_summary_{c,d,l,s}` variants for code, Doppler, phase-locked and signal-strength observations). These store the post-processing of each RINEX file.

- *Auxiliary*, embargoes, documents, licenses, logs, and the node/data-centre infrastructure tables; together with a small `authentication` table used by the legacy admin endpoints.

One small but instructive design choice. The `country` table uses the ISO three-letter code (`iso_code`) as its natural primary key instead of a surrogate `id`, and `station.country_code` is a direct `character(3)` foreign key onto it. This single decision removes the most common "which agency are you talking about" join ambiguity from station queries and is a representative example of the small simplifications the schema makes possible.

Spatial data: today and tomorrow. Station coordinates are stored as plain `numeric(12,3)` for the Cartesian `x/y/z` components and as `numeric(6,4)` / `numeric(7,4)` for the geodetic `lat/lon`. The PostGIS extension is *not* currently installed, the only non-default extension is `plpgsql`. Bounding-box and polygon checks are implemented in Java by the helpers of the `Geometry` package (Section 5.3), which is sufficient for present needs and keeps the database side neutral. Migrating those checks to PostGIS is one of the items in the project's future-work list (Chapter 6), but is expected to make little difference to correctness; the motivation, when it comes, will be query plan quality on large datasets.

Indexing today and indexing tomorrow. The schema today carries about twenty-five explicit indexes, almost all of them B-tree indexes on foreign-key columns (`idx_file` on `rinex_file(id_station)`, `idx_qc_filesum` on `qc_report_summary(id_rinexfile)` with `fillfactor=90` to favour reads over writes, and so on). This handles the common access patterns of the API, where every endpoint joins from the `station` or `rinex_file` side through a chain of foreign keys.

The natural next step, and the main reason the larger `DB_3.0_F` fixture was provisioned, is to validate the introduction of **BRIN** (Block Range Index) indexes on the date columns of `rinex_file` (`reference_date`, `published_date`, `release_date`) and of the `highrate` variant. BRIN indexes are appropriate here because RINEX files are inserted in approximate date order, so the on-disk row layout already correlates with the indexed column, which is the precondition under which a BRIN index is small and fast. The differential test framework introduced in Chapter 5 plays a role in this evaluation: by running the same diff cases against the `DB_3.0_F` fixture before and after the BRIN addition, the framework gives a concrete

behavioural signal in addition to whatever `EXPLAIN ANALYZE` would say in isolation.

3.5 Application and CI Servers

3.5.1 Payara Server

Payara Server [5] is a derivative of the GlassFish reference implementation of Jakarta EE, maintained as a commercial distribution by Payara Services Ltd. The laboratory adopted Payara several years ago and the production API is deployed onto an existing Payara installation; this project did not change that choice. Two specific properties of Payara matter for the work reported here. First, Payara Server packages a full Jakarta EE runtime, which is what the production WAR is built against. Second, Payara distributes a companion product, **Payara Micro**, which is a single, self-contained JAR that can be launched from the command line with a WAR on its argument list:

```
java -jar payara-micro.jar --port 8081 target/ROOT.war
```

Payara Micro is the engine on which the HTTP-level differential harness relies: the harness boots two instances of Payara Micro on two different ports, one serving the baseline WAR and the other the **HEAD!** WAR, and replays the same requests against both.

3.5.2 Apache Tomcat as an Alternative

Apache Tomcat [30] is the most widely used servlet container in the Java world, and is sometimes used as a lightweight alternative to a full Jakarta EE server. It was considered for the second prototype but ruled out because, unlike Payara, Tomcat does not ship a full Jakarta EE 11 runtime, CDI, JPA and Bean Validation would have had to be wired in by hand. Standardising on Payara across the three projects eliminated that overhead.

3.5.3 Maven as Build Tool

All three applications are built with **Apache Maven**. The `pom.xml` of the production API configures three plugins worth highlighting: `maven-war-plugin`, which produces the deployable `ROOT.war`; `maven-failsafe-plugin`, which runs integration tests (named `*IT.java`) during the `verify` phase and emits **JUnit!** (**JUnit!**) XML reports; and `maven-dependency-plugin`, which downloads the Payara Micro JAR into `target/payara-micro/` during the build so that the HTTP harness can later spawn it as a subprocess.

3.5.4 GitLab and GitLab Runner

GitLab [17] was already discussed in Chapter 2 as the CI/CD platform adopted by the laboratory. From the point of view of the technology stack, two components are relevant. The *GitLab server* is a self-managed installation [19] running on a Virtual Machine (VM) inside the laboratory; it hosts the repository, the issues, the pipeline definitions and the merge requests. The *GitLab Runner* [18] is a separate process, hosted on another VM (tagged `ubi-runner-.31`), that polls the GitLab server for pending jobs and executes them inside a Docker container. That separation is significant: it means the runner machine is the one that needs network access to the laboratory's database, not the GitLab server itself.

3.6 Version Control: Git and GitLab

Git [31] is the version control system used across all three applications. Beyond the obvious use as a source code repository, Git is used in two non-trivial ways in this project. First, the CI pipeline of the production API uses `git worktree` to materialise an additional copy of the repository pinned to a baseline commit, on the runner's local file system; this is the mechanism by which the differential harness obtains the "previous version" against which the current build is compared. Second, the harness identifies the baseline as the first parent of `HEAD` (`HEAD~1`), which makes the comparison meaningful even on long-running refactor branches in which the developer commits frequently. The exact pattern is discussed in detail in Section 5.4.

GitLab itself plays the role of the centralised Git server. All three repositories are hosted under `gitlab.com`: the production API under the `segalubi` group, and the two prototypes under the personal namespace of the author.

3.7 Containerization: Docker and docker-compose

Docker [32] is used in two distinct roles in this project. The first role is as a runtime for the CI jobs: each GitLab CI job declared in this project runs inside a Docker container based on the official `maven:3.9.6-eclipse-temurin-21` image, which guarantees a clean and reproducible build environment regardless of what is installed on the runner host. The second role is as a local development aid: the two prototypes ship a `docker-compose.yaml` [33] that stands up a PostgreSQL instance, and, in the case of the first prototype, a pair of

Payara Server containers for the dual-node Arquillian tests described in Section 4.4.

For the production API, Docker is intentionally *not* used to package the application: the laboratory's deployment model is to copy the WAR onto an existing Payara installation, and a container image would not bring any benefit while adding an additional moving part. This is briefly revisited as a possible future evolution in Chapter 6.

3.8 Network Access: The Laboratory VPN

The production database of the laboratory is reachable on the internal network address `192.168.168.30:5432` and is not exposed to the public Internet. Two consequences follow. First, the GitLab Runner that executes the differential tests against the real database must itself be inside the laboratory's network, this is why the runner is tagged `ubi-runner-.31` and pinned to a specific machine. Second, developers working from outside the laboratory have to connect to the network through a VPN configuration maintained by the laboratory staff (Luís Carvalho).

The technical choice of VPN technology is outside the scope of this project, the laboratory uses an existing solution, but modern alternatives such as WireGuard [34] were considered briefly as a lighter-weight option for one-off remote testing sessions.

3.9 Conclusion

The technology stack on which this project is built is in large part imposed by its environment: Jakarta EE on Payara, PostgreSQL on the internal network, GitLab as the self-managed CI platform. Where there was room for choice, between Jakarta EE and Spring Boot, between Payara Server and Apache Tomcat, between PostgreSQL and MySQL, between JUnit 5 and TestNG, the decisions documented in this chapter were guided by a preference for fewer moving parts, by the alignment with what the laboratory already runs in production and by the desire to keep the differential testing harness as simple as possible. The next chapter describes the iterative methodology by which these technologies were combined into a working solution.

Chapter

4

Methodology and Architecture

4.1 Introduction

This chapter describes the methodology by which the testing infrastructure of the project was developed and then explains its overall architecture. The methodology is openly iterative: rather than building the final solution directly on top of the production API, two intermediate prototype applications were written first, each addressing a separate concern in isolation, and the lessons learnt from them were carried over into the production application. Section 4.2 situates the work against the legacy API that motivated the refactor. Section 4.3 describes the iterative methodology. Sections 4.4 and 4.5 discuss the two prototypes, `java-ee-app` and `syncspace-jakarta-app`. Finally, Section 4.6 gives a high-level view of the architecture of the differential testing framework that is detailed at the code level in Chapter 5.

4.2 The Legacy API and the Need for a Safety Net

The starting point of the work is, to be blunt about it, a code base that had grown organically for several years and that nobody fully understood any more. The previous version of the EPOS API contained Java methods of more than five thousand lines, source files of more than thirty thousand lines, and many subtle behavioural quirks introduced as patches by successive developers in response to specific bug reports. The corollary was that no member of the laboratory could realistically guarantee, for an arbitrary change, that no client of the API would be silently broken. Manual testing covered a few well-known requests and tacitly relied on the expectation that nobody would do anything too creative.

The full rewrite of the API on top of Jakarta EE 11 made the internal structure much cleaner, short methods, well-bounded JAX-RS resource classes, clear separation between parsing, validation, query construction and serialization, but it also opened a window of risk: at any given commit during the refactor, an endpoint may produce subtly different output from the version deployed in production. The role of this project is to build the safety net that turns those subtle differences into a visible, investigable signal in the CI pipeline.

4.3 Iterative Methodology

The work was organised into three iterations, each carried out in its own dedicated repository. This separation served two purposes. First, it kept each iteration small and focused: the first prototype is deliberately stripped down to two trivial endpoints, so that the entire complexity is in the CI configuration and not in the application; the second prototype is the opposite, focusing on application code and leaving CI aside; the production API then brings the two strands back together. Second, the prototypes have value in their own right as didactic artifacts that future students of the laboratory can read and re-use.

The three iterations are summarised in Table 4.1.

Iteration	**Focus**	**What was carried over**
Prototype 1 `java-ee-app`	CI/CD pipeline patterns: dual Payara, Arquillian, JUnit XML reporting.	The shape of the `.gitlab-ci.yml`, the Maven cache strategy and the choice of base Docker image.
Prototype 2 `syncspace-jakarta-app`	Jakarta EE 11 features: CDI, JAX-RS, JPA, Bean Validation, MicroProfile JWT, exception mappers.	Validation patterns, exception mapper layout, record-based DTOs, some of which were adopted by the refactor of the production API, some only used didactically.
Production `api_v4.0`	End-to-end integration: full pipeline, code-level diff, HTTP-level diff against real database via VPN.	Final artifact of the project.

Table 4.1: The three iterations of the project and their respective focus.

4.4 Prototype 1: java-ee-app

The first prototype is a deliberately minimal Jakarta EE application whose role is to validate the CI/CD pipeline. It exposes a single JAX-RS application class with two trivial endpoints, a `Hello World` text endpoint and a small POJO serialised to JSON, and a pair of Arquillian integration tests that exercise them against a managed Payara container.

4.4.1 Layout and Stack

The application is a Maven WAR project targeting Java SE 17 and Jakarta EE 11. Its dependencies, beyond the Jakarta EE API bundle (declared with `provided` scope), are JUnit 5 Jupiter and the Arquillian `payara-server-managed` adapter. The `src/test` folder contains an `AbstractBaseIT.java` that prepares a ShrinkWrap deployment and a `JsonApplicationIT.java` that issues HTTP requests against the deployed application.

4.4.2 The CI Pipeline

The interesting part of this prototype is the `.gitlab-ci.yml`, shown in Listing 4.1. The pipeline has two stages: `build`, which produces the WAR, and `test`, which runs the integration tests against *two* Payara Server containers, declared as *services* in the job definition. The dual-server setup was useful as a stress test of the runner network: if the pipeline can boot two containers and run tests against them, it can certainly boot the single Payara Micro instance used by the production pipeline.

```
default:
  image: maven:3.9.6-eclipse-temurin-21
  tags:
    - ubi-runner

cache:
  key: global-maven-cache
  paths:
    - .m2/repository

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/
     repository"

stages:
  - build
  - test
```

```yaml
build-job:
  stage: build
  script:
    - mvn clean package -DskipTests
  artifacts:
    paths:
      - target/*.war

integration-test-job:
  stage: test
  services:
    - name: payara/server-full:7.2026.5
      alias: payara-node-1
    - name: payara/server-full:7.2026.5
      alias: payara-node-2
  script:
    - sleep 45
    - mvn verify
  artifacts:
    when: always
    reports:
      junit:
        - "target/failsafe-reports/failsafe-summary.xml"
        - "target/failsafe-reports/TEST-*.xml"
    paths:
      - target/failsafe-reports/
    expire_in: 1 week
```

Code Listing 4.1: `.gitlab-ci.yml` of the first prototype, two-stage pipeline with dual Payara services.

Three patterns from this prototype were carried over to the production pipeline:

1. the pinned base image and the `ubi-runner` tag, which together guarantee that the job runs on a known runner with a known toolchain;
2. the global Maven cache, which avoids re-downloading the repository on every job;
3. the `reports: junit:` stanza, which is what makes GitLab understand failed assertions as failed pipeline steps rather than as opaque exit codes.

The forty-five-second `sleep` that precedes `mvn verify` compensates for the time required by the Payara containers to become reachable; a cleaner health-poll loop was tried and discarded as not worth the additional complexity for a prototype.

4.5 Prototype 2: syncspace-jakarta-app

The second prototype goes in the opposite direction. Where the first prototype is a few lines of application code with a complete CI pipeline, the second prototype is a complete application with no CI pipeline at all. Its goal is to exercise the Jakarta EE 11 specifications listed in Section 3.3 on a domain that is small enough to fit in a student's head but large enough to require non-trivial use of each specification.

4.5.1 The Domain

`syncspace-jakarta-app` is the back end of a hypothetical room-booking service for a co-working space. The domain is centred on three entities: `User`, `Room` and `Booking`. A room has a type (`HOT_DESK`, `PODCAST_STUDIO`, `BOARDROOM`, `EVENT_SPACE`), a maximum capacity and an active flag. A booking links a user to a room over a time interval and counts the number of attendees. Users authenticate through a login endpoint that issues a JSON Web Token (JWT); subsequent calls to the booking endpoints are protected by MicroProfile JWT [35].

4.5.2 Jakarta Specifications Exercised

Each part of the application was deliberately written to use a specific Jakarta or MicroProfile specification:

- **JAX-RS 4.0 [36].** The `SyncSpaceApplication` class declares an `@ApplicationPath("/api/v1")` and the resources (`AuthResource`, `BookingResource`, `RoomResource`) follow standard JAX-RS idioms, with `@Inject` for service dependencies and the `Response` builder for fluent response construction.
- **CDI 4.1.** Services and exception mappers are CDI beans, injected into the JAX-RS resources. `AuthResource` is `@RequestScoped`; services are implicitly singletons.
- **JPA 3.2 [27].** Entities use `@Entity`, `@ManyToOne`, `@OneToMany`, `@Enumerated(STRING)` for the enums, and `@PrePersist` for default timestamps.
- **Bean Validation 3.1.** A custom `@ValidDateRange` class-level annotation is paired with a validator that ensures `endTime` is strictly greater than `startTime` and attaches the error to the correct property through `addPropertyNode("endTime")`.

- **MicroProfile JWT 2.1.** The `@LoginConfig(authMethod = "MP-JWT")` on the application class enables token-based authentication; tokens are signed manually with Nimbus JOSE+JWT using an RSA signature, with the user identifier as the subject and the user's roles as the `groups` claim.

- **Exception mappers.** Four dedicated mappers (`BusinessValidationMapper`, `ValidationExceptionMapper`, `JsonProcessingExceptionMapper`, `GlobalExceptionMapper`) translate domain and framework exceptions into structured JSON responses, avoiding the leaking of internal stack traces to the API consumer. The end-to-end flow is summarised in Figure 4.1.

4.5.3 What was Carried Over

Most of the code of this prototype was never reused in the production API, the production domain is very different from a room booker. Two patterns were nevertheless adopted into the refactor. The first is the use of a small set of focused JAX-RS exception mappers rather than a single catch-all JAX-RS `ExceptionMapper<Throwable>`: this leaves HTTP status codes meaningful and gives the differential test harness a stable signal to assert on. The second is the use of `record`-based POJOs for request and response bodies, which avoids hand-written `equals`/`hashCode` and makes diffing two responses simpler.

The remaining features, in particular the full JWT flow, were retained in the prototype as a self-contained reference example to which future students can look for guidance on Jakarta EE 11 idioms.

4.6 Architecture of the Differential Testing Framework

The architecture of the final testing framework, applied to the production API, is summarised in Figure 4.2. It is made of three concentric layers: the GitLab pipeline, the harness processes, and the application instances themselves.

The flow is as follows. A push by the developer is received by the self-managed GitLab server, which schedules a pipeline on the runner tagged `ubi-runner-.31`. The pipeline goes through four jobs across three stages: `build-job` produces the **HEAD!** WAR; `integration-test-job` runs the regular JUnit integration tests against that WAR; `validate-diff-job` reconstructs both baseline WARs (one from `HEAD~1`, one from the fixed commit

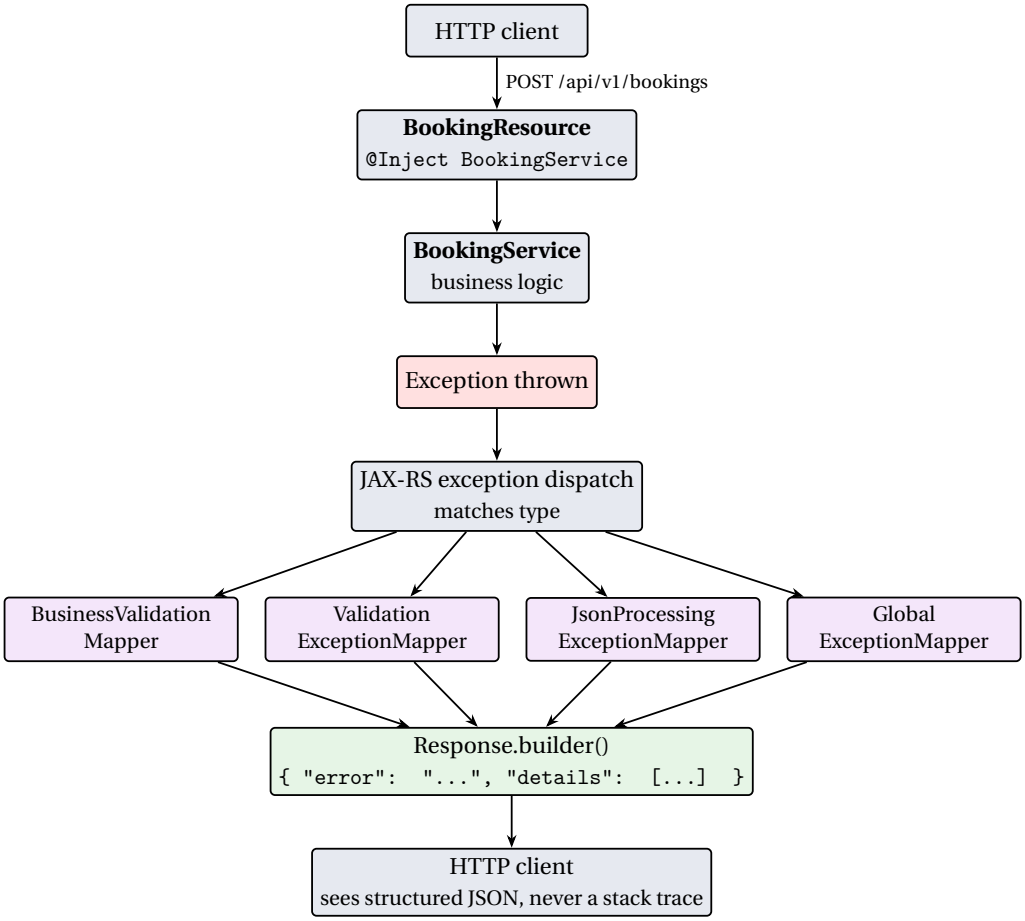

Figure 4.1: The CDI + JAX-RS exception-handling flow used by `syncspace-jakarta-app` and partly inherited by `api_v4.0`. Any exception thrown inside a resource or service bean is intercepted by the most specific registered `ExceptionMapper`, which produces a stable JSON error body.

in `$DIFF_FIXED_BASELINE`) using Git worktrees and runs the code-level differential harness against each in turn; `http-diff-job` boots Payara Micro instances of all three WARs on different ports, connects them to the real database over the laboratory's network and replays the test cases listed in `http-diff-cases.json` against all of them, collecting per-case diffs against both baselines independently. The two diff jobs publish reports that GitLab understands both as JUnit-style results and as raw artifacts for later inspection.

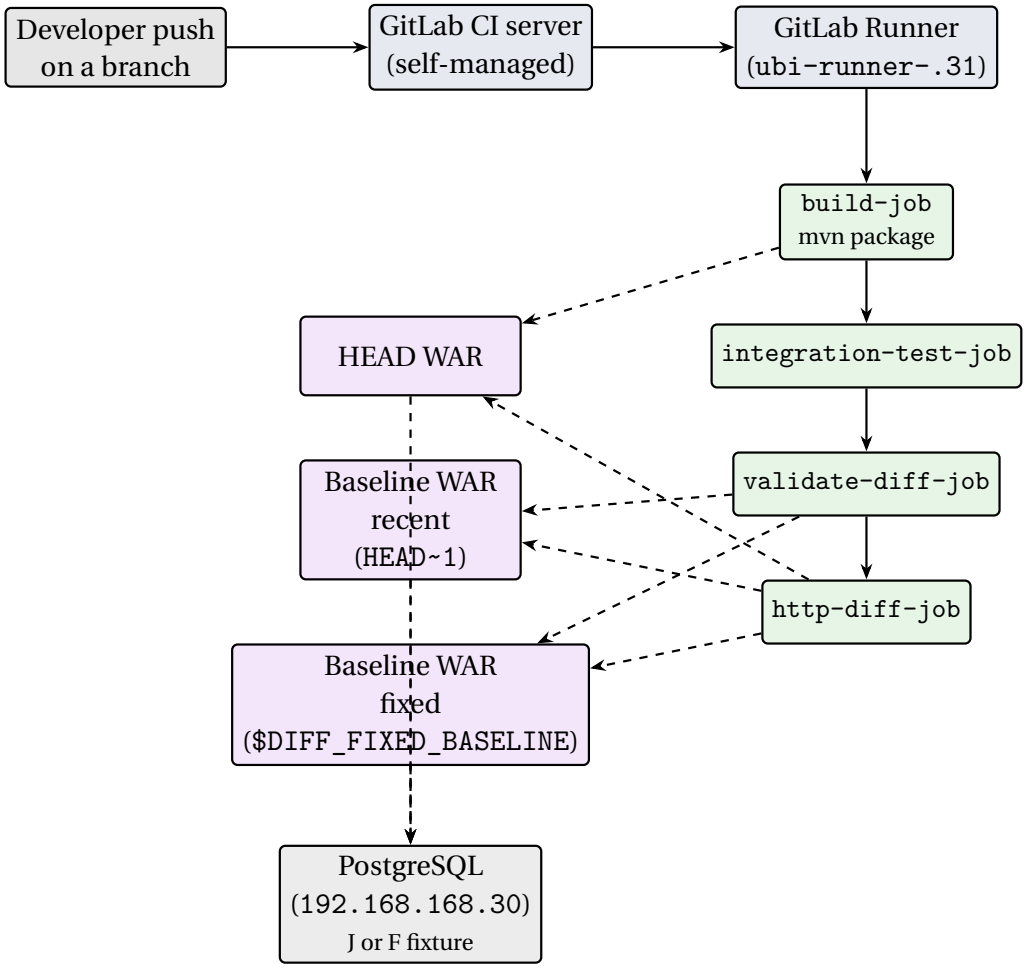

Figure 4.2: Overall architecture of the differential testing framework. Solid arrows denote control flow inside the pipeline; dashed arrows denote artifact or data access. Both diff jobs run a three-way comparison between the **HEAD!** build and two baselines, the immediate parent commit and a fixed reference commit, all pointed at the same database fixture.

4.7 Conclusion

The iterative methodology adopted in this project, two prototypes followed by the production application, traded an apparently longer time-to-result for a much sharper separation of concerns between CI/CD infrastructure and Jakarta EE feature usage. The first prototype gave the `.gitlab-ci.yml` its shape; the second prototype gave the production refactor a small but useful set of Jakarta EE idioms. The next chapter applies the resulting framework to

the production API and walks through its CI pipeline, the two diff harnesses and their reports.

Chapter

5

Use Case: EPOS API

5.1 Introduction

This chapter describes the application of the framework introduced in Chapter 4 to the production API of the laboratory, `api_v4.0`. Section 5.2 explains briefly the EPOS context that the API serves. Section 5.3 walks through the structure of the application itself, with emphasis on the elements that the testing framework has to be aware of (resource classes, output formats, persistence). Section 5.4 dissects the `.gitlab-ci.yml` file, job by job. Sections 5.5 and 5.6 document the two diff harnesses, code-level and HTTP-level, including their inputs, outputs and reporting contract. Section 5.7 discusses how the reports are consumed and the kinds of issues the harnesses have surfaced in practice. Section 5.8 concludes.

5.2 The EPOS Context

The EPOS [1] is a pan-European research infrastructure in solid Earth science, organised into Thematic Core Services for seismology, near-fault observatories, volcano observations, geomagnetic observations, anthropogenic hazards and GNSS data and products [2], among others. The SEGAL laboratory of UBI contributes to the GNSS Thematic Core Service by operating a service node that hosts station metadata and RINEX [3] observation files. The production API described in this chapter is the public interface of that service node.

From the point of view of the test framework, the salient properties of this domain are three. First, the entities are relatively static (new stations are added rarely, file metadata grows over time but in well-defined ways), which

makes a snapshot of the database a stable baseline against which to compare. Second, the typical client request returns a long list of records, which means that any behavioural divergence produces a long diff and the comparison algorithm must avoid being overwhelmed by it. Third, the same information is offered in multiple serialization formats, which multiplies the surface area to be tested but does not change the underlying logic.

5.3 Architecture of `api_v4.0`

5.3.1 Source Layout

The application is a Maven WAR project targeting Java SE 21 and Jakarta EE 11, deployed onto Payara Server 6 with `finalName=ROOT` (so that it is reachable at the root context `/`). Its source tree is organised into four top-level packages, summarised in Table 5.1.

Package	Purpose
`Config`	Database connection bootstrap (`Loader.java`, persistence-unit factory).
`DB_Tables`	The forty JPA entity classes that model the relational schema (stations, networks, agencies, file types, observation summaries, embargos, ...).
`Geometry`	Geometric validation, in particular the native polygon checks used by the bounding-box queries.
`Glass`	The REST layer proper: JAX-RS resources (`StationEndpoints`, `FilesEndpoints`, `ListEndpoints`, `Start`), parsers, format writers (JSON/XML/CSV/script) and the validation/shape helpers (`Validate.java`, `Shape.java`).

Table 5.1: Top-level packages of the production API.

The four packages cooperate as a classical layered architecture (Figure 5.1): the `Glass` layer exposes the HTTP surface, the validation and geometry helpers form a thin business layer, the JPA entities of `DB_Tables` act as the persistence layer, and `Config` is a cross-cutting concern that handles bootstrap.

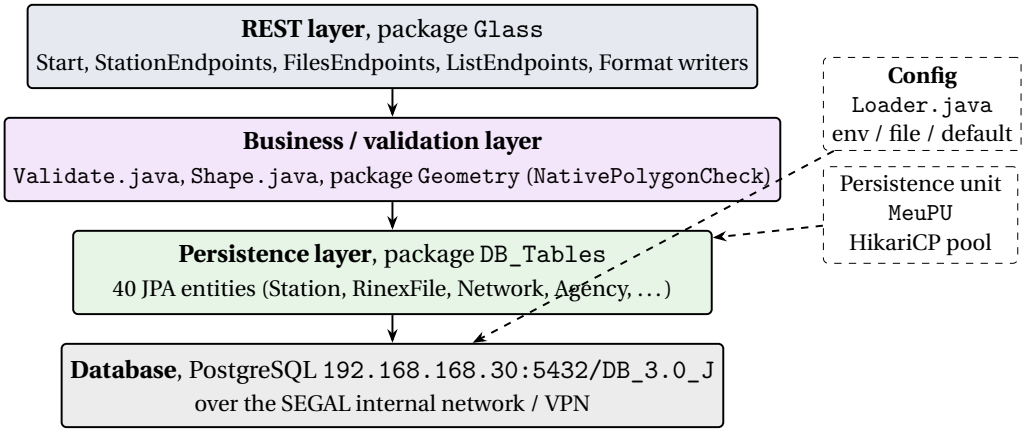

Figure 5.1: Layered architecture of api_v4.0. Solid arrows denote ordinary call/data flow; dashed arrows denote configuration and infrastructure binding established at deployment time.

5.3.2 Endpoints and Output Formats

The REST surface is declared in four classes inside the Glass package. Start.java carries the @ApplicationPath("/api") annotation and registers a PingResource for the trivial /api/ping liveness check used by both the diff harness and the deploy verification of http-diff-job. The three substantive resources are listed in Table 5.2.

Every endpoint accepts a {format} path parameter and emits the response in JSON, XML, CSV or, for the file endpoints, a downloadable shell script. The same OpenAPI 3.1.0 document is served at /swagger.html through a Swagger UI hosted inside the WAR.

Figure 5.2 traces the end-to-end path of a single request through the Payara process, from the HTTP listener down to the database and back, calling out the Jakarta specifications involved at each step.

5.3.3 Persistence

The persistence unit, declared in src/main/resources/META-INF/persistence.xml, is named MeuPU (inherited from the legacy code). It is configured as RESOURCE_LOCAL, with Hibernate as the JPA provider and a connection pool managed by HikariCP. Database connection parameters, host, port, database name, user, password, are resolved at startup by Config/Loader.java, with the following precedence: environment variables, then a Config.config file at the root of the deployment, then hard-coded defaults pointing at the laboratory's database (192.168.168.30:5432).

Resource	Base path	Approximate scope
`StationEndpoints`	`/api/stations`	Seven endpoints for station metadata, indexed by station identifier, location, agency, network, equipment and coordinates.
`FilesEndpoints`	`/api/files`	Four endpoints for RINEX file metadata, by agency, antenna, bounding box and date; by marker; high-rate; and an aggregated combination.
`ListEndpoints`	`/api/list`	A parametric endpoint that returns the valid values of a given filter (agencies, networks, countries, equipment types, file types).

Table 5.2: The three substantive JAX-RS resources of `api_v4.0`.

The laboratory maintains two database fixtures with the same schema but different volumes of data:

- `DB_3.0_J` hosts a small but representative production subset, on the order of ten stations, eighty-six networks, two hundred and thirty agencies and roughly one hundred and forty-six thousand `rinex_file` rows at the time of writing. This is the fixture used by the correctness diff runs, because it is small enough to allow per-case comparison of full response bodies without overwhelming the report.
- `DB_3.0_F` hosts the full dataset and is used by the volume-oriented diff runs, where the goal is not byte-for-byte equality but the detection of changes that would only be apparent at scale, for example, an inadvertent `N+1` query introduced by a refactor, or a query plan regression visible only on a large table.

The precedence rules of `Loader.java` are what allow the `http-diff-job` to switch the running WARs between the two fixtures without rebuilding the artifact: the job sets the appropriate environment variables in its `variables:` block and the application picks them up at boot.

The full schema, as introspected from the running database, contains forty-two tables in the `public` schema and around fifty foreign-key con-

straints, organised into a few clear groups: core domain (`station`, `network`, `agency`, `country`, `rinex_file`, `highrate_rinex_file`, `file_type`), junction tables (`agency_network`, `station_network`, `station_agency_role`, `node_station`, `node_data_center`), quality-control summaries (`qc_report_summary`, `qc_constellation_summary`, `qc_header_observations`, `qc_navigation_msg` and four `qc_observation_summary_{c,d,l,s}` variants), and auxiliary tables for embargoes, documents, logs, licenses and infrastructure. A simplified view of the central entities is given in Figure 5.3; the chapter on technologies (Section 3.4.4) discusses the design of the database itself in more detail.

5.4 The CI/CD Pipeline

The pipeline of `api_v4.0` extends the patterns validated on the first prototype with two additional jobs in a new `diff` stage. Each diff job compares the current build against *two* baselines, in what amounts to a three-way comparison: the immediate parent commit `HEAD~1` (which catches regressions introduced by the commit currently being pushed) and a *fixed* reference commit identified by a SHA stored in the `DIFF_FIXED_BASELINE` GitLab CI/CD variable (which catches the slow drift that accumulates over many commits along a long-running refactor branch). The full file is shown in Listing 5.1 and Listing 5.2.

```
default:
  image: maven:3.9.6-eclipse-temurin-21
  tags:
    - ubi-runner-.31

cache:
  key: global-maven-cache
  paths:
    - .m2/repository

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/
     repository"
  GIT_DEPTH: "20"

stages:
  - build
  - test
  - diff

build-job:
  stage: build
  script:
```

```
    - mvn clean package -DskipTests
  artifacts:
    paths:
      - target/*.war

integration-test-job:
  stage: test
  script:
    - mvn verify
  artifacts:
    when: always
    reports:
      junit:
        - "target/failsafe-reports/failsafe-summary.xml"
        - "target/failsafe-reports/TEST-*.xml"
    paths:
      - target/failsafe-reports/
    expire_in: 1 week
```

Code Listing 5.1: Header, build and integration test jobs of the production pipeline.

The pinned runner tag, the cached Maven repository and the `maven:3.9.6-eclipse-temurin-21` base image are direct descendants of the first prototype. Two additions are worth calling out. The `GIT_DEPTH: "20"` variable bumps the default shallow-clone depth from one to twenty commits, which is the minimum required for the diff jobs to be able to inspect `HEAD~1` without an additional `git fetch`. The third stage, `diff`, hosts the two new jobs.

```
validate-diff-job:
  stage: diff
  needs:
    - job: build-job
      artifacts: false
  variables:
    DIFF_FIXED_BASELINE: ""   # commit SHA, set in GitLab CI
      /CD Variables
  script:
    - |
      run_against_baseline() {
        local LABEL="$1"; local BASELINE_SHA="$2"
        if [ -z "$BASELINE_SHA" ]; then
          echo "No $LABEL baseline available - skipping"
          return 0
        fi
        local WORK="/tmp/${LABEL}-baseline-build"
        git worktree remove --force "$WORK" 2>/dev/null ||
          true
```

```
        git worktree prune
        rm -rf "$WORK"
        git worktree add "$WORK" "$BASELINE_SHA"
        ( cd "$WORK" && mvn -q -o package -DskipTests )
        export BASELINE_WAR="$WORK/target/ROOT.war"
        export DIFF_REPORT_DIR="target/diff-report/$LABEL"
        export DIFF_LABEL="$LABEL"
        mkdir -p "$DIFF_REPORT_DIR"
        mvn test -Dtest=DiffHarness -DfailIfNoTests=false
      }
      RECENT_SHA=$(git rev-parse --verify HEAD~1 2>/dev/null
          || echo "")
      run_against_baseline recent "$RECENT_SHA"
      run_against_baseline fixed  "$DIFF_FIXED_BASELINE"
  artifacts:
    when: always
    paths:
      - target/diff-report/
    reports:
      junit:
        - "target/surefire-reports/TEST-Diff.DiffHarness.xml
            "
    expire_in: 4 weeks

http-diff-job:
  stage: diff
  needs:
    - job: build-job
      artifacts: true
  variables:
    DATABASE_URL: "postgres://postgres:postgres@192
       .168.168.30:5432/DB_3.0_J"
    JAKARTA_SCHEMA_ACTION: "none"
    DIFF_FIXED_BASELINE: ""
  script:
    - |
      build_baseline() {
        local LABEL="$1"; local BASELINE_SHA="$2"
        local WORK="/tmp/${LABEL}-http-baseline-build"
        git worktree remove --force "$WORK" 2>/dev/null ||
            true
        git worktree prune
        rm -rf "$WORK"
        git worktree add "$WORK" "$BASELINE_SHA"
        ( cd "$WORK" && mvn -q -o package -DskipTests )
        echo "$WORK/target/ROOT.war"
      }
      RECENT_SHA=$(git rev-parse --verify HEAD~1 2>/dev/null
          || echo "")
```

```
      [ -n "$RECENT_SHA" ]             && export
         BASELINE_RECENT_WAR=$(build_baseline recent "
         $RECENT_SHA")
      [ -n "$DIFF_FIXED_BASELINE" ]    && export
         BASELINE_FIXED_WAR=$(build_baseline fixed  "
         $DIFF_FIXED_BASELINE")
      mvn test -Dtest=HttpDiff -DfailIfNoTests=false
  artifacts:
    when: always
    paths:
      - target/http-diff-report/
    reports:
      junit:
        - "target/surefire-reports/TEST-Diff.HttpDiff.xml"
    expire_in: 4 weeks
```

Code Listing 5.2: `validate-diff-job` and `http-diff-job`, the three-way differential testing jobs.

A few design decisions visible in the two listings deserve a comment.

Two baselines, one job. Each diff job runs the harness twice, once per baseline, with the report written into a separate subdirectory of `target/diff-report/` (or `target/http-diff-report/`). The *recent* baseline is the immediate parent of `HEAD`, which keeps the per-commit comparison focused on the change just introduced and avoids the avalanche of diffs that would otherwise occur on long-running refactor branches. The *fixed* baseline is a commit ID stored in the GitLab variable `DIFF_FIXED_BASELINE`: it acts as a long-term reference against which the cumulative drift of the branch is measured, and is typically pointed at the last commit known to have shipped to the EPOS portal. The two baselines surface different classes of regressions (one immediate, one accumulated) without doubling the maintenance cost of test fixtures.

Worktree-based baseline build. The baseline WAR is built by checking out the baseline commit into a separate Git worktree (`/tmp/baseline-build`) and invoking `mvn -q -o package -DskipTests` there. Worktrees were chosen over the more obvious `git checkout` because they preserve the current working directory untouched: the harness can freely compare files from both checkouts without having to alternate between branches.

Defensive skip per baseline. Each baseline is treated independently. If `HEAD~1` does not exist (the trivial case of the first commit of the repository), the *recent* comparison is skipped. If `DIFF_FIXED_BASELINE` is unset, which

is the expected state at the very start of a refactor branch, before a fixed reference has been chosen, the *fixed* comparison is skipped. A pipeline can therefore have zero, one or two diff sub-runs depending on the situation, and never fails simply because a baseline is unavailable.

Job dependencies. validate-diff-job does not need the artifact of build-job, it rebuilds the current WAR as a side effect of mvn test, and is therefore declared with needs: artifacts: false to avoid an unnecessary artifact download. http-diff-job, on the other hand, requires the current WAR to spawn Payara Micro and so declares artifacts: true.

5.5 The Code-Level Diff: DiffHarness.java

DiffHarness.java is a JUnit 5 test that compares two versions of a specific Java class on a shared input grid, in process. Its core idea is to load the baseline and the **HEAD!** WARs through two independent URLClassLoader instances, to reflect into the class under test on both sides and to invoke each method of interest with the same arguments. Any difference between the two returned values, thrown exceptions included, is recorded as a row in the diff report.

The pipeline invokes the harness once per baseline (recent and fixed; see Section 5.4). The exported environment variables BASELINE_WAR, DIFF_REPORT_DIR and DIFF_LABEL tell the harness which WAR to load against **HEAD!** and where to write its outputs. The on-disk layout is target/diff-report/recent/ and target/diff-report/fixed/, so a glance at the artifact tree of a finished pipeline immediately distinguishes a per-commit regression from a long-term drift.

The harness focuses on two classes for which behaviour is critical and for which an entire endpoint cannot be exercised in isolation: Validate, which parses and validates client query parameters, and Shape, which evaluates geometric polygon queries against the bounding box of a request. Both classes are purely functional, take simple inputs (strings, lists of doubles) and return simple outputs (booleans, lists, error strings), which makes them well suited to a code-level diff.

The output of each sub-run is twofold. A **JUnit!** XML report (TEST-Diff.DiffHarness.xml, with the DIFF_LABEL appearing as a test-suite property) integrates with GitLab so that divergences appear as failed assertions on the pipeline page. In parallel, a richer CSV/JSON report under the per-baseline subdirectory records every input row, the value returned by the baseline, the value returned by **HEAD!** and an indicator of whether the two match. The CSV format makes the report easy to open in a spreadsheet for ad-hoc analysis.

5.6 The HTTP-Level Diff: `HttpDiff.java`

`HttpDiff.java` is the second harness, and the one that exercises the API end-to-end. Its execution proceeds in four phases, summarised graphically in Figure 5.4. Unlike the code-level harness, which is invoked twice by the pipeline, the HTTP harness runs once and internally boots *all* the available WARs in parallel (the **HEAD!** WAR, the recent baseline if any, and the fixed baseline if any). It then issues every test case against every running instance and emits one diff report per baseline.

Phase 1, Boot. The harness spawns up to three child Java Virtual Machine (JVM) processes from the Payara Micro JAR that the Maven build had previously downloaded into `target/payara-micro/`. The current WAR (`target/ROOT.war`) is deployed on port 8080. If the `BASELINE_RECENT_WAR` environment variable is set, the recent baseline is deployed on port 8081; if `BASELINE_FIXED_WAR` is set, the fixed baseline is deployed on port 8082. All processes read their database configuration from the environment variables injected by the `http-diff-job`, which means they all point at the same fixture (typically `DB_3.0_J` for correctness runs).

Phase 2, Deploy verification. Before any test case is sent, the harness polls `GET /api/ping` on each active port until every running instance returns `200 OK`, with a generous time-out. This handshake was added in response to spurious early failures observed during development, where the first request of a test would happen to land on a still-deploying container; the corresponding commit message in the repository history is the literal sentence "*ci: HttpDiff, prove deploy actually happened before counting it as ready*".

Phase 3, Test replay. The list of test cases lives in `src/test/resources/http-diff-cases.json`, a small JSON array in which each entry describes an HTTP request (method, path, headers, optional body). Listing 5.3 shows the shape of the file.

```
[
  {
    "name": "ping",
    "method": "GET",
    "path": "/api/ping",
    "headers": { "Accept": "application/json" }
  },
  {
    "name": "station-short-json",
    "method": "GET",
```

```
    "path": "/api/stations/station/short/json",
    "headers": { "Accept": "application/json" }
  }
]
```

Code Listing 5.3: Excerpt from `http-diff-cases.json`, showing two of the test cases used by `HttpDiff`.

For each case, the harness fires the request against every active port in parallel, captures status code, response headers and body, and then performs a three-way comparison: **HEAD!** against the recent baseline (if present) and **HEAD!** against the fixed baseline (if present). The two comparisons are kept separate so that "a change my commit introduced" and "a change that has accumulated since the fixed reference" remain distinguishable in the report. Headers irrelevant to behavioural equivalence (`Date`, `Server`, `Content-Length`) are explicitly ignored. The body is compared as a normalised string, sorted JSON object keys, trimmed whitespace, in order to avoid spurious diffs that would mask the real ones.

Phase 4, Tear-down and reporting. After the last case, all child processes are killed and the harness emits two CSV/JSON reports, one per baseline, under `target/http-diff-report/recent/` and `target/http-diff-report/fixed/`. A combined JUnit XML report under `target/surefire-reports/` flags each diff as a failed test case, tagged with its baseline label, so that GitLab shows the regressions both on the merge-request page and on the pipeline page.

5.7 Reports, Operations and Findings

Each pipeline run publishes four artifact paths that survive for between one and four weeks: the integration-test reports, the validate-diff report, the HTTP-diff report, and the JUnit XML files consumed by GitLab itself. The retention windows are deliberately asymmetric: integration test results expire faster because they are cheap to reproduce, while diff reports are kept longer because they are the historical record of behavioural change during the refactor.

During the development of the framework the harnesses surfaced a handful of real regressions that had been introduced by what looked like cosmetic refactors. The most representative example is the extraction of latitude and longitude range bounds to named constants, which was reverted after the diff harness reported a silent behavioural change in the boundary handling, the corresponding commits in the repository history are `7c8ff37 (refactor: extract lat/lon range bounds to named constants)`

and `cc223ba (Revert "refactor: extract lat/lon range bounds to named constants")`. From the point of view of this project, those two commits are the best validation that the framework does the job it was designed for: it caught a regression that no human reading of the diff was likely to catch by eye.

5.8 Conclusion

The application of the framework to `api_v4.0` demonstrates that a small, focused set of CI jobs is sufficient to provide a useful safety net during a long-running refactor of a Jakarta EE API, provided that the harnesses are designed around the specifics of the system: a moving baseline, multiple response formats, an internal database. The next chapter draws the main conclusions of the project and outlines the items that, by choice or by constraint, were left out of its scope.

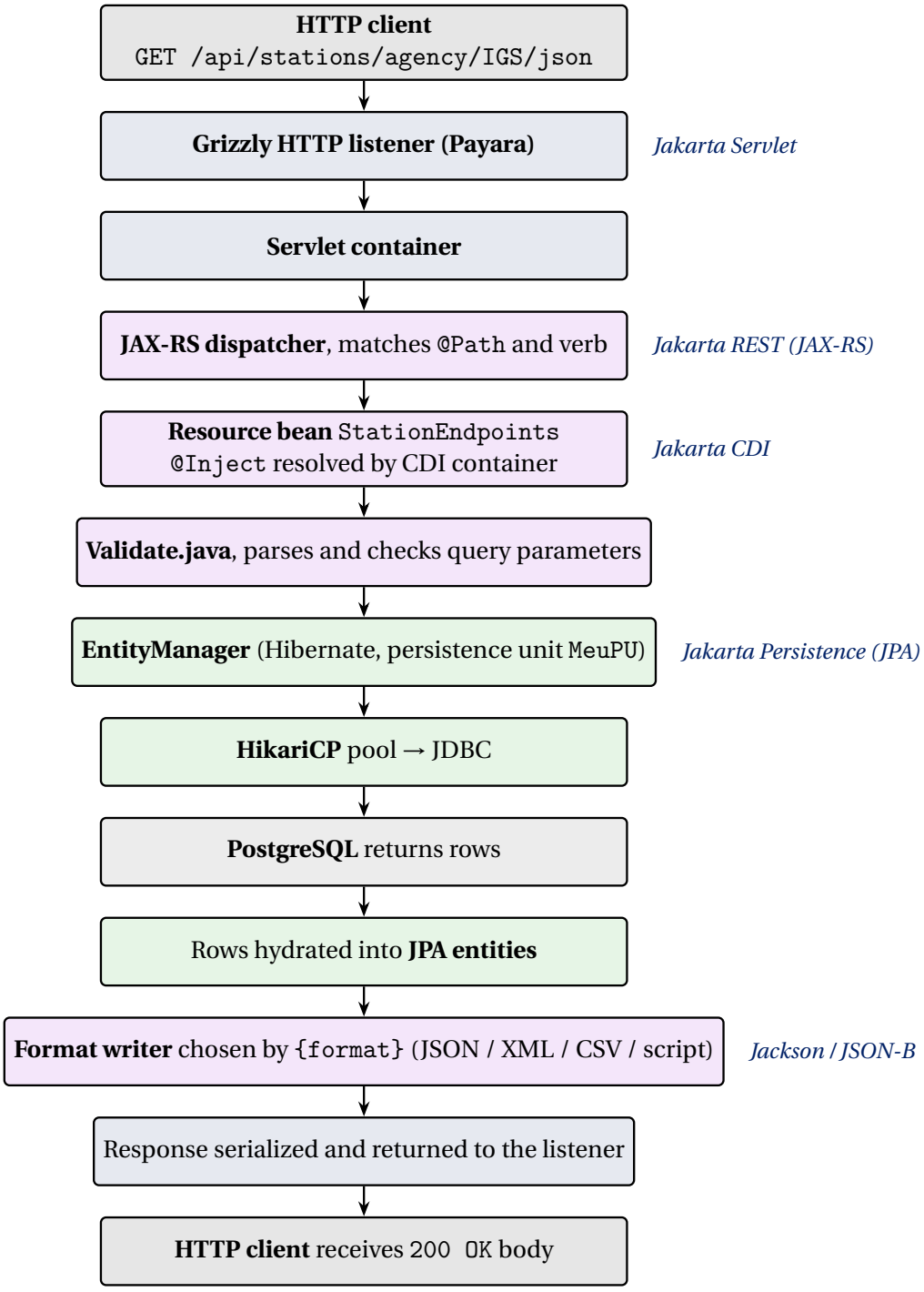

Figure 5.2: End-to-end request lifecycle of a JAX-RS call to `api_v4.0` on Payara. The right column names the Jakarta specification active at each step.

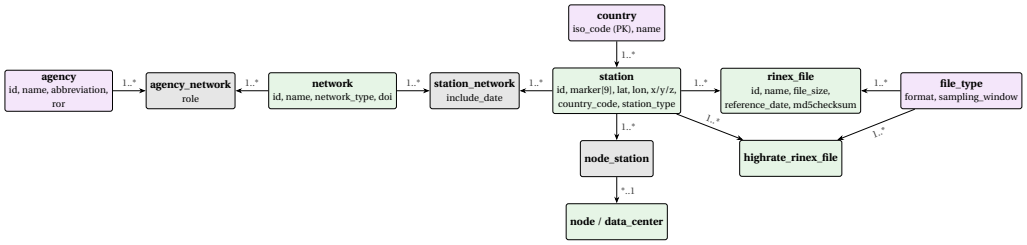

Figure 5.3: Simplified view of the core EPOS data model, with representative columns as they appear in the live PostgreSQL schema. Green entities carry domain data; purple entities are controlled-vocabulary lookups (note that `country.iso_code` is the natural primary key, not a surrogate `id`); grey entities are junction tables that materialise the many-to-many relationships of the model. Arrows point from the "one" side to the "many" side of the relationship.

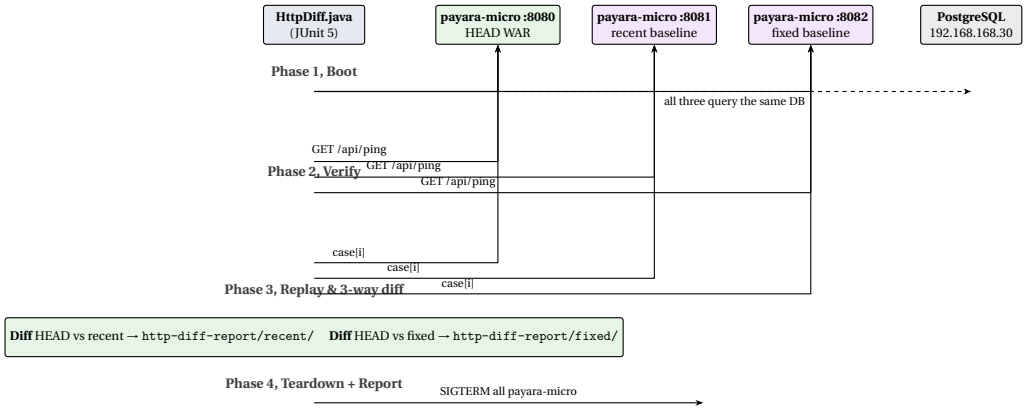

Figure 5.4: Phases of `HttpDiff.java`. The harness boots one Payara Micro per available WAR, the **HEAD!** build and whichever of the recent/fixed baselines were provided, waits for all of them to be ready, fires every test case against every instance in parallel, then performs a three-way comparison and emits one diff report per baseline. All instances share the same real PostgreSQL fixture over the laboratory VPN.

Chapter

6

Conclusions and Future Work

6.1 Main Conclusions

The work described in this report set out to address a very concrete problem of the SEGAL laboratory: how to refactor a large, several-years-old Jakarta EE API without silently breaking the EPOS clients that depend on it. The answer proposed and built in the course of this project takes the form of a three-stage, CI/CD-driven testing framework that for every commit produced by the refactor compares the resulting build against the immediately preceding one, both at the level of internal Java classes and at the level of the HTTP responses produced by the API.

Several conclusions emerge from the work.

Differential testing fits a refactor naturally. Asserting that a given input produces a given output is a brittle contract during a refactor, because every legitimate cosmetic change to a response field requires updating dozens of fixtures. Asserting that the new build produces *the same* output as the old one sidesteps the problem entirely: a legitimate cosmetic change appears once in the diff report, can be approved, and the next pipeline run treats it as the new baseline. This is the property that allowed the framework to be useful from the first commit it ran on, without a long ramp-up period of fixture creation.

Two layers of comparison are better than one. The code-level harness (`DiffHarness.java`) and the HTTP-level harness (`HttpDiff.java`) catch different classes of regressions. The first catches behavioural changes in focused helper classes (`Validate`, `Shape`) before they propagate into endpoint responses, with very fast feedback and without the cost of booting an

application server. The second catches integration-level regressions that only manifest once a real request hits a real database. Either one in isolation would have left blind spots; together they cover the spectrum that the laboratory cares about.

A self-hosted pipeline is not optional in this context. Because the production database lives on the internal network of the laboratory and is not exposed to the public Internet, only a runner that itself sits inside that network can exercise the API end-to-end against it. The decision to self-manage both the GitLab control plane and the GitLab Runner host therefore followed from the data-locality constraint, not from a preference for self-hosting per se. The same architecture would not be appropriate for an open-source project served by gitlab.com's shared runners.

The two prototypes paid for themselves. In retrospect, having spent the first few weeks of the project on `java-ee-app`, a deliberately trivial application whose only purpose was to harden the pipeline, and then on `syncspace-jakarta-app`, a deliberately fuller application whose only purpose was to exercise Jakarta EE 11 features in isolation, saved significant time later, when the production API had to be addressed. Each prototype answered a small, well-bounded set of questions in an environment in which mistakes were cheap. Without them, every one of those questions would have had to be answered against the production code base, where mistakes are not.

The framework reveals what a human review would miss. The clearest evidence of value, beyond any abstract argument, is the list of commits in the repository in which a refactor was proposed, the diff harness flagged a behavioural change, the change was investigated, and the refactor was reverted. The most representative example, discussed in Section 5.7, is the named-constants refactor of the latitude/longitude bounds: a textbook clean-up that introduced a subtle off-by-one in the boundary check and that no reasonable amount of code review would have caught at the rate the laboratory commits. That alone is, in the author's view, the justification for the framework.

6.2 Future Work

A number of items were deliberately left out of the scope of this project, either because they were not strictly necessary to meet the objectives, or because

they would have expanded the project beyond what could reasonably be addressed in the time available.

OpenAPI-driven test case generation. The list of HTTP cases exercised by `HttpDiff.java` is currently maintained by hand in `http-diff-cases.json`. The OpenAPI 3.1.0 document shipped inside the WAR already describes every endpoint and the shape of its parameters; a future evolution of the framework could derive the case list automatically from the specification, ensuring that newly added endpoints are exercised by the diff from the moment they are merged, with no need to update the test fixture.

Schema diffing for the OpenAPI document itself. A different but adjacent use of the same document is to diff the OpenAPI of the baseline against that of the **HEAD!**: any breaking change to the contract (a removed endpoint, a changed type, a tightened validation) would be reported as a separate kind of regression. This would complement the behavioural diff with a contract diff.

Containerization of the production API. The production application is currently deployed by copying the WAR onto an existing Payara installation. Packaging the application as a Docker image, with a Payara base image and a small amount of entry-point configuration, would simplify the migration to a containerised deployment platform if the laboratory ever chooses to move in that direction, and would also remove the ambiguity between "the WAR that was tested" and "the WAR that was deployed" that occasionally arises with the current model.

Performance regression testing and BRIN indexing. The current diff harnesses focus on behavioural equivalence and ignore response time. A natural extension is to record the wall-clock duration of each HTTP case for both baselines and **HEAD!** across both database fixtures (`DB_3.0_J` and `DB_3.0_F`) and to flag cases in which the new version is significantly slower than the old one. The threshold would necessarily be statistical rather than absolute, because the runner shares resources with other jobs; a small benchmarking subsystem would be required. The first concrete consumer of such a subsystem is already lined up: the planned introduction of BRIN indexes on the date columns of `rinex_file` and `highrate_rinex_file` (Section 3.4.4) needs exactly this kind of empirical evidence to justify itself against the larger `F` fixture.

PostGIS migration for spatial queries. Bounding-box and polygon checks are currently performed in Java by the helpers of the `Geometry` package. Mi-

grating those checks to native PostGIS operators would let the database planner reason about spatial selectivity and is the natural complement to the BRIN work above. The expectation is that correctness would not change significantly (the existing checks are already faithful to the specification) and that the main benefit would be query plan quality on the `DB_3.0_F` fixture, which makes this an ideal target for the same differential framework: the same diff cases should pass byte-for-byte against the PostGIS-enabled version, and only the timing distribution should change.

Snapshot testing of representative responses. Not every endpoint is amenable to a moving baseline; some responses should remain stable across many releases (for example, the `/api/ping` response, or the OpenAPI document itself). A small set of snapshot tests, in addition to the differential ones, would protect those invariants and would explicitly mark them as "known stable" rather than "allowed to drift".

Generalisation to other laboratory services. The framework has been designed against the specifics of `api_v4.0` but its architecture is not specific to that API: the dual-Payara-Micro approach, the worktree-based baseline, the JSON-driven case list and the **JUnit!**/CSV reporting could in principle be reused by any other Jakarta EE service of the laboratory. Reorganising the harness code into a small library, with the application-specific bits factored into a clean configuration interface, would make this reuse straightforward, and is a natural next step for a continuation of the work.

Hardening of operational aspects. Finally, several operational aspects of the framework deserve more work than was possible inside the scope of this project: failure recovery in the harness (retries on transient network errors, clean termination of orphan Payara Micro processes), more granular VPN routing, automated provisioning of the GitLab Runner host through configuration management, and a richer dashboard on top of the diff reports collected over time. None of these are blocking for the day-to-day use of the framework, but each would make it more comfortable to live with for the years to come.

Overall, the framework delivered by this project addresses the problem that motivated it, has already proved its value on a real regression caught during the refactor, and lays the groundwork for the laboratory to continue evolving the API with a much higher degree of confidence than was previously possible. The items listed in this section are natural next steps; none of them invalidates the design that was put in place.

Bibliography

[1] EPOS ERIC. European Plate Observing System — EPOS, 2024. [Online] https://www.epos-eu.org/. Accessed June 2026.

[2] EPOS GNSS Data and Products Thematic Core Service. EPOS GNSS Data Gateway, 2024. [Online] https://gnss-epos.eu/. Accessed June 2026.

[3] Werner Gurtner and Lou Estey. RINEX — The Receiver Independent Exchange Format, Version 3.05, 2020. International GNSS Service (IGS), RINEX Working Group.

[4] Eclipse Foundation. Jakarta EE 11 Platform Specification, 2024. [Online] https://jakarta.ee/specifications/platform/11/. Accessed June 2026.

[5] Payara Services Ltd. Payara Platform Community Documentation, 2024. [Online] https://docs.payara.fish/. Accessed June 2026.

[6] JUnit Team. JUnit 5 User Guide, 2024. [Online] https://junit.org/junit5/docs/current/user-guide/. Accessed June 2026.

[7] Cédric Beust. TestNG documentation, 2024. [Online] https://testng.org/. Accessed June 2026.

[8] AtomicJar, Inc. Testcontainers for Java, 2024. [Online] https://java.testcontainers.org/. Accessed June 2026.

[9] Red Hat, Inc. Arquillian Container Testing Framework, 2024. [Online] https://arquillian.org/. Accessed June 2026.

[10] REST Assured Contributors. REST Assured — Java DSL for testing REST services, 2024. [Online] https://rest-assured.io/. Accessed June 2026.

[11] Postman, Inc. Postman API Platform, 2024. [Online] https://www.postman.com/. Accessed June 2026.

[12] Postman, Inc. Newman — a Postman command-line collection runner, 2024. [Online] `https://github.com/postmanlabs/newman`. Accessed June 2026.

[13] Twitter Engineering. Diffy — Automatically catch bugs in service-oriented architectures, 2015. [Online] `https://blog.twitter.com/engineering/en_us/a/2015/diffy-testing-services-without-writing-tests`. Accessed June 2026.

[14] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

[15] Jenkins Project. Jenkins — The leading open source automation server, 2024. [Online] `https://www.jenkins.io/`. Accessed June 2026.

[16] GitHub Inc. GitHub Actions Documentation, 2024. [Online] `https://docs.github.com/actions`. Accessed June 2026.

[17] GitLab Inc. GitLab CI/CD documentation, 2024. [Online] `https://docs.gitlab.com/ci/`. Accessed June 2026.

[18] GitLab Inc. GitLab Runner documentation, 2024. [Online] `https://docs.gitlab.com/runner/`. Accessed June 2026.

[19] GitLab Inc. GitLab Self-Managed — Install and configure, 2024. [Online] `https://about.gitlab.com/install/`. Accessed June 2026.

[20] Roy T. Fielding. Architectural Styles and the Design of Network-based Software Architectures. In *Ph.D. dissertation, University of California, Irvine*, 2000. Chapter 5 introduces REST.

[21] OpenAPI Initiative. OpenAPI Specification, Version 3.1.0, 2021. [Online] `https://spec.openapis.org/oas/v3.1.0`. Accessed June 2026.

[22] SmartBear Software. Swagger UI, 2024. [Online] `https://swagger.io/tools/swagger-ui/`. Accessed June 2026.

[23] Oracle Corporation. *Java Platform, Standard Edition 21 API Specification*. Oracle Corporation, 2023. [Online] `https://docs.oracle.com/en/java/javase/21/`.

[24] VMware, Inc. Spring Boot Reference Documentation, 2024. [Online] `https://docs.spring.io/spring-boot/index.html`. Accessed June 2026.

[25] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, 2024. [Online] https://www.postgresql.org/docs/16/. Accessed June 2026.

[26] Oracle Corporation. MySQL 8.0 Reference Manual, 2024. [Online] https://dev.mysql.com/doc/refman/8.0/en/. Accessed June 2026.

[27] Eclipse Foundation. Jakarta Persistence 3.2, 2024. [Online] https://jakarta.ee/specifications/persistence/3.2/. Accessed June 2026.

[28] Red Hat, Inc. Hibernate ORM Documentation, 2024. [Online] https://hibernate.org/orm/documentation/. Accessed June 2026.

[29] Brett Wooldridge. HikariCP — A high-performance JDBC connection pool, 2024. [Online] https://github.com/brettwooldridge/HikariCP. Accessed June 2026.

[30] The Apache Software Foundation. Apache Tomcat documentation, 2024. [Online] https://tomcat.apache.org/. Accessed June 2026.

[31] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014. [Online] https://git-scm.com/book/en/v2.

[32] Docker, Inc. Docker Documentation, 2024. [Online] https://docs.docker.com/. Accessed June 2026.

[33] Docker, Inc. Compose Specification, 2024. [Online] https://compose-spec.io/. Accessed June 2026.

[34] Jason A. Donenfeld. WireGuard: Next Generation Kernel Network Tunnel, 2020. [Online] https://www.wireguard.com/. Accessed June 2026.

[35] Eclipse Foundation. Eclipse MicroProfile, 2024. [Online] https://microprofile.io/. Accessed June 2026.

[36] Eclipse Foundation. Jakarta RESTful Web Services 4.0, 2024. [Online] https://jakarta.ee/specifications/restful-ws/4.0/. Accessed June 2026.